

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – MINI APPLICATION DEVELOPMENT (CHATBOT /
SUMMARIZER / TRANSLATOR)

Course Code:	CSA65	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO4 – Precision (BL3, BL4,BL5)	Assessment:	Assessment Tool 1-Mini Application Development (Chatbot / Summarizer / Translator)
Weightage:	20% of CIA	Total Marks:	2 * 50= 100
CO4 Statement:	<i>Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.(BL3, BL4,BL5)</i>		
Student Name:	S.Sirivally	Reg. No.:	192472371
Section / Batch:	2024-CSE(AI)	Date:	19-08-26

Instructions to Students

- **Answer any 2 questions. Each question carries 50 marks. Total: 100 marks.**
- Implement the solution using **Python**.
- Select and justify an appropriate AI/LLM model or technique.
- Develop a **working application with a user interface**.
- Test the application using multiple input cases.
- Demonstrate handling of invalid, empty, or unexpected inputs.
- Clearly explain the application architecture and workflow.
- Submit the source code, test results, screenshots/demo, and documentation.
- Explain the limitations of the selected model/technique and suggest possible improvements.

Code of Conduct

I ___S.Sirivally / 192472371___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ___Sirivally.S_____

SECTION A – MINI APPLICATION DEVELOPMENT (CHATBOT / SUMMARIZER / TRANSLATOR)

TEXT SUMMARIZATION APPLICATIONS

1. Student Notes Summarizer – Build an application that summarizes lengthy study notes into important points. The system should accept text as input and generate a concise summary suitable for examination revision.

Problem Statement

Students often have lengthy textbooks, lecture notes, and study materials that are difficult to revise before examinations. The objective of this application is to develop an AI-powered Student Notes Summarizer that accepts academic study material and converts it into concise, examination-oriented revision material using Google Gemini.

Objectives

- Develop a practical Generative AI application using Python and Google Gemini.
- Summarize lengthy academic study material into concise revision notes.
- Accept study material through direct text input or file/document upload.
- Allow the user to specify the subject/course and academic level.
- Generate structured output containing important concepts and examination takeaways.
- Handle empty, invalid, unexpected, and API-related errors.
- Test the application using multiple input cases.
- Provide a simple and user-friendly interface.

AI Model and Technique

Google Gemini is used as the Large Language Model. It was selected because it provides strong natural-language understanding and generation capabilities and is suitable for educational summarization. Prompt engineering is used to instruct the model to preserve important technical concepts, remove unnecessary repetition, and generate examination-oriented content.

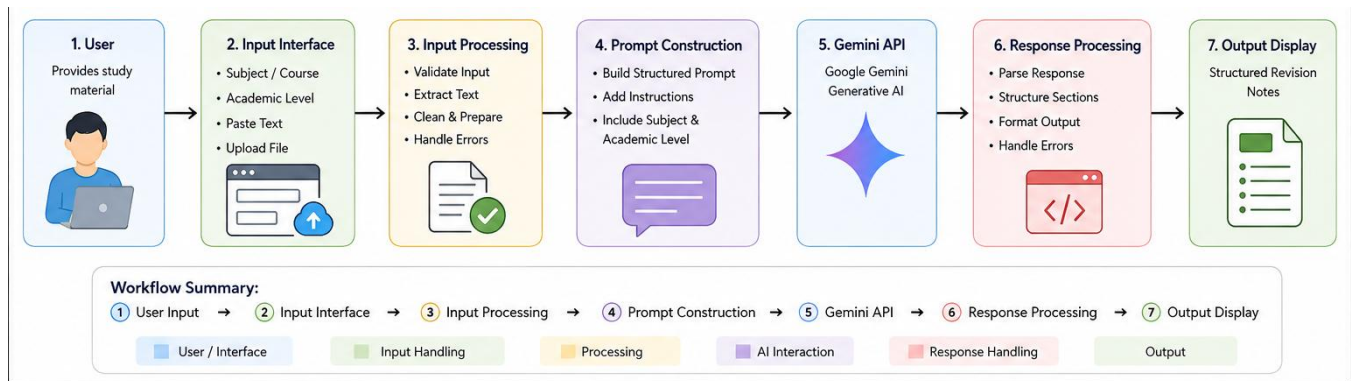
Technologies and Tools Used

- Python
- Google Gemini API
- Google GenAI SDK
- Web-based user interface
- Web browser

Application Features

- Subject/course input.
- Academic-level selection.
- Paste Text input method.
- Upload File input method.
- Input validation.
- Gemini-powered summarization.
- Structured examination revision output.
- Summary, key concepts, important points, exam takeaways, terminology, and possible exam questions.
- Clear and user-friendly error messages.

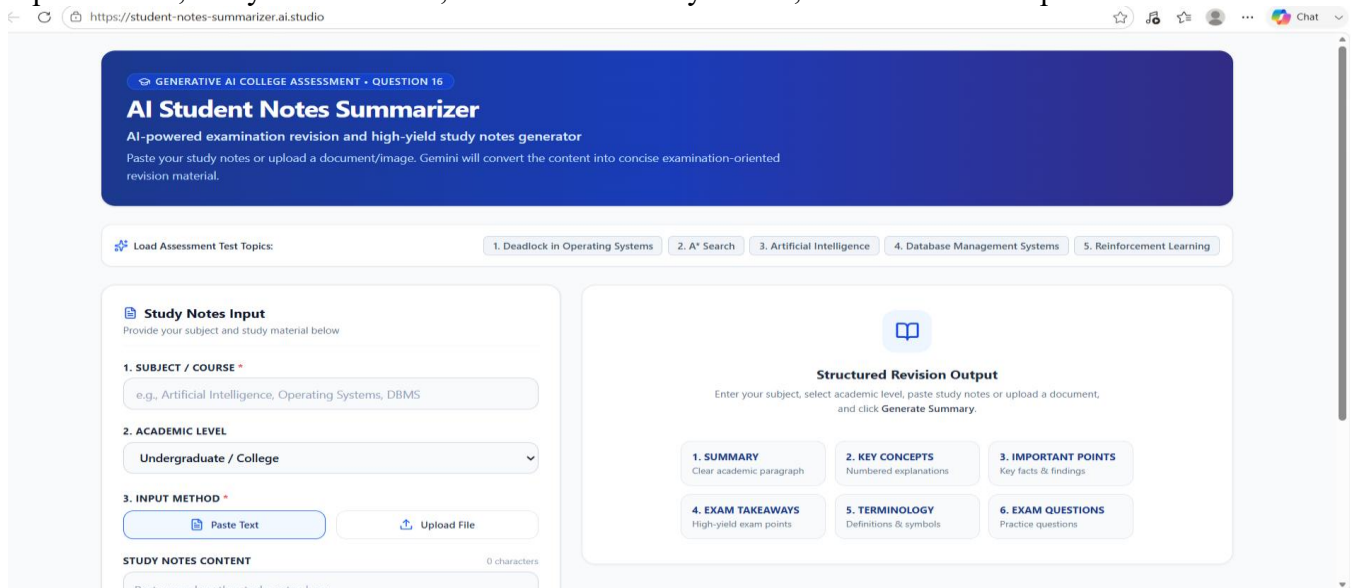
Application Architecture and Workflow



The user provides the subject, academic level, and study material. The application validates the input, constructs a structured prompt, sends it to Google Gemini, processes the response, and displays the revision material.

User Interface

The application uses a single main page containing the subject/course field, academic-level selection, input method, study-material area, Generate Summary button, and structured output area.



Caption: User interface of the AI Student Notes Summarizer.

Pseudocode

```
BEGIN
Initialize Google Gemini API
Display Student Notes Summarizer
INPUT subject/course
INPUT academic level
SELECT input method
IF Paste Text is selected
    READ study notes
ELSE IF Upload File is selected
    READ uploaded file
    EXTRACT available content
END IF
IF input is empty
    DISPLAY validation message
    STOP
END IF
Construct summarization prompt
Send prompt to Gemini API
IF request succeeds
    Display structured revision material
ELSE
    Display user-friendly error
END IF
END
```

Representative Source Code

The complete source code should be submitted separately. The following shows the Gemini API interaction.

```
from google import genai

client = genai.Client(api_key=API_KEY)

response = client.models.generate_content(
    model="YOUR_SUPPORTED_GEMINI_MODEL",
    contents=prompt
)

result = response.text
```

The API key must not be hard-coded in a publicly shared source file. Store it securely using an environment variable or secret configuration.

Test Cases

Test Case	Input	Expected Result	Status
TC01	Deadlock in Operating Systems notes	Structured revision summary	PASS

TC02	Artificial Intelligence notes	Concise academic summary	PASS
TC03	A* Search notes	Summary with $f(n)$, admissibility, consistency, optimality and complexity	PASS
TC04	Short machine learning note	Short meaningful summary	PASS
TC05	Empty input	Validation message requesting study material	PASS

Test Case Screenshots

TC01	Deadlock in Operating Systems notes
------	-------------------------------------

Output:

Study Notes Input

Provide your subject and study material below

1. SUBJECT / COURSE *

Operating Systems

2. ACADEMIC LEVEL

School

3. INPUT METHOD *

Paste Text

Upload File

STUDY NOTES CONTENT

1,453 characters

A deadlock is an undesirable state in an operating system where two or more processes are permanently blocked because each holds a resource while waiting for another resource held by some other process in the group.

Coffman Conditions for Deadlock:
Deadlock occurs if and only if all four conditions hold simultaneously:

1. Mutual Exclusion: At least one non-shareable resource exists that can only be held by one process at a time.

Generate Summary

Clear

Examination Revision Material

Subject: Operating Systems

Level: School

Clean Plain-Text

Copy

Download .txt

Print

1 SUMMARY

A deadlock represents a problematic situation in an operating system where multiple processes become stuck in a permanent waiting state. This occurs because every process in the group is holding a specific resource while simultaneously requesting another resource that is currently occupied by a different process in the same group. Because none of the processes can proceed or release their held resources, the system reaches a standstill that requires specific management strategies to resolve or prevent.

2 KEY CONCEPTS

1. Mutual Exclusion: A condition where a resource is non-shareable and can be accessed by only one process at any given time.

2. Hold and Wait: A state where a process maintains possession of its current resources while waiting to acquire additional resources held by others.

3. No Preemption: A rule stating that resources cannot be taken away from a process by force; they must be released voluntarily.

4. Circular Wait: A scenario where a chain of processes exists such that each process is waiting for a resource held by the next process in the cycle.

3 IMPORTANT POINTS

1. Deadlock only occurs if all four Coffman conditions are present at the same time.

2. Deadlock prevention focuses on breaking at least one of the four necessary conditions.

3. The Banker Algorithm is a common method used for deadlock avoidance by checking if a state is safe before granting resources.

4. The Ostrich Algorithm is a practical approach that ignores deadlocks because they occur infrequently in many systems.

4 EXAM TAKEAWAYS

1. Memorize the four Coffman conditions as they are the foundational requirements for a deadlock to exist.

2. Understand that breaking the circular wait condition is a primary method for preventing deadlocks.

3. Distinguish between prevention, which stops deadlocks from ever occurring, and detection, which identifies them after they happen.

4. Be prepared to explain why the Ostrich Algorithm is used in general-purpose operating systems despite the theoretical risk of deadlock.

5 IMPORTANT TERMINOLOGY

1. Resource Allocation Graph: A visual tool used to track which processes hold or request specific resources to detect cycles.

2. Safe State: A condition where the system can allocate resources to each process in some order without entering a deadlock.

Output:**Study Notes Input**

Provide your subject and study material below

1. SUBJECT / COURSE *

Artificial Intelligence

2. ACADEMIC LEVEL

Postgraduate

3. INPUT METHOD *

Paste Text

Upload File

STUDY NOTES CONTENT

989 characters

- Problem Solving via Search: Uninformed search (Breadth-First Search, Depth-First Search) and informed search (A* search, Minimax with Alpha-Beta pruning).

- Machine Learning Paradigms: Supervised learning (mapping inputs to labels), unsupervised learning (clustering and dimensionality reduction), and reinforcement learning (learning policies via environmental feedback).

- Intelligent Agents: Systems that perceive their environment via sensors and act upon that environment through actuators according

Generate Summary

Clear

Examination Revision Material

Subject: Artificial Intelligence

Level: Postgraduate

Copy

Download .txt

Print

Clean Plain-Text

1 SUMMARY

Artificial Intelligence encompasses the computational study of systems designed to emulate human cognitive functions including reasoning, perception, and autonomous decision-making. At the postgraduate level, the field integrates formal logic for knowledge representation, heuristic-driven search algorithms for optimization, and diverse machine learning paradigms to facilitate adaptive behavior. By utilizing the agent-based model, these systems interact with dynamic environments through sensors and actuators, striving to maximize performance metrics through rational action selection and environmental feedback loops.

2 KEY CONCEPTS

1. Intelligent Agents: Autonomous entities that utilize the PEAS framework (Performance measure, Environment, Actuators, Sensors) to perceive inputs and execute actions to achieve specific goals.
2. Search Algorithms: Computational methods for navigating state spaces, categorized into uninformed strategies like Breadth-First Search and informed strategies like A* search that utilize heuristics.
3. Machine Learning Paradigms: Methodologies for pattern recognition and decision-making, specifically supervised learning, unsupervised learning, and reinforcement learning.
4. Knowledge Representation: The formalization of information using structures such as first-order logic, ontologies, and semantic networks to enable machine reasoning.

3 IMPORTANT POINTS

1. Rational agents are defined by their ability to select actions that maximize expected performance based on their percept history.
2. Informed search algorithms such as Minimax with Alpha-Beta pruning are essential for optimizing decision-making in adversarial environments.
3. Supervised learning focuses on mapping input data to predefined labels, whereas unsupervised learning identifies hidden structures like clusters within unlabeled data.
4. Reinforcement learning relies on environmental feedback to refine policies, distinguishing it from static supervised or unsupervised approaches.

4 EXAM TAKEAWAYS

1. Be prepared to differentiate between uninformed search strategies and informed search strategies based on their use of heuristic functions.
2. Understand the components of the PEAS framework and how they define the operational scope of an intelligent agent.
3. Memorize the fundamental distinctions between supervised, unsupervised, and reinforcement learning paradigms.
4. Recognize the role of first-order logic and ontologies in structuring knowledge for automated reasoning systems.

5 IMPORTANT TERMINOLOGY

1. Heuristic: A technique used in informed search to estimate the cost to reach a goal, improving efficiency

Output:

Study Notes Input

Provide your subject and study material below

1. SUBJECT / COURSE *

Artificial Intelligence

2. ACADEMIC LEVEL

Undergraduate / College

3. INPUT METHOD *

Paste Text

Upload File

STUDY NOTES CONTENT

1,063 characters

A* Search (A-Star) is an informed graph traversal and pathfinding algorithm combining Dijkstra's Algorithm with Greedy Best-First Search.

Evaluation Function:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the exact path cost from the start node to node n , $h(n)$ is the estimated heuristic cost from n to the goal, and $f(n)$ is the estimated total cost of the cheapest path through n .

Generate Summary

Clear

Examination Revision Material

Subject: Artificial Intelligence

Level: School

Clean Plain-Text

Copy

Download .txt

Print

1 SUMMARY

A-Star search is an informed pathfinding algorithm used to find the shortest path between a starting point and a goal. It functions by evaluating nodes based on a combination of the distance already traveled and a heuristic estimate of the remaining distance. By balancing these two values, the algorithm efficiently explores the search space to identify the most cost-effective route while ensuring that the solution found is optimal under specific conditions.

2 KEY CONCEPTS

1. Evaluation Function: The formula $f(n) = g(n) + h(n)$ used to rank nodes, where $g(n)$ is the cost from the start and $h(n)$ is the estimated cost to the goal.
2. Admissibility: A property of a heuristic where it never overestimates the actual cost to reach the goal, ensuring the algorithm finds the shortest path.
3. Consistency: A property where the estimated cost from a node is no greater than the step cost to a neighbor plus the estimated cost from that neighbor to the goal.

3 IMPORTANT POINTS

1. A-Star combines the strengths of Dijkstra's Algorithm, which considers path cost, and Greedy Best-First Search, which considers heuristic estimates.
2. Admissibility is a requirement for the algorithm to guarantee an optimal solution in tree search scenarios.
3. Consistency ensures that the algorithm does not need to revisit nodes already placed in the closed list, which improves efficiency in graph searches.
4. The primary limitation of A-Star is its high memory usage, as it must store all generated nodes in memory.

4 EXAM TAKEAWAYS

1. Memorize the evaluation function $f(n) = g(n) + h(n)$ and be prepared to calculate $f(n)$ for given nodes.
2. Understand that the time and space complexity are both $O(b^d)$, where b is the branching factor and d is the depth of the solution.
3. Distinguish between admissibility and consistency, noting that consistency is a stronger condition required for graph search optimality.
4. Recognize that memory consumption is the main bottleneck for this algorithm in large search spaces.

5 IMPORTANT TERMINOLOGY

1. Heuristic: An educated guess or estimate used to guide the search process toward the goal.
2. Branching Factor: The number of successors or children that each node has in the search tree.
3. Optimal Path: The path that has the lowest total cost among all possible paths to the goal.
4. Open List: A data structure used to store nodes that have been generated but not yet expanded.

6 POSSIBLE EXAM QUESTIONS

1. How does the evaluation function $f(n) = g(n) + h(n)$ influence the pathfinding process in A-Star search?
2. Why is memory consumption considered the primary limitation of the A-Star algorithm?
3. What is the difference between an admissible heuristic and a consistent heuristic?
4. If a heuristic overestimates the cost to the goal, does A-Star still guarantee an optimal solution?

Output:

 Study Notes Input

Provide your subject and study material below

1. SUBJECT / COURSE *

Reinforcement Learning

2. ACADEMIC LEVEL

Undergraduate / College

3. INPUT METHOD *

 Paste Text Upload File

STUDY NOTES CONTENT

1,039 characters

interacting with an environment to maximize cumulative numerical reward over time.

Markov Decision Process (MDP) Framework:

An MDP is formally defined by the tuple (S, A, P, R, γ) :

- S : State space
- A : Action space
- P : Transition probability function $P(s' | s, a)$
- R : Immediate reward function $R(s, a, s')$

 Generate Summary

Clear

 Examination Revision Material

Subject: Reinforcement Learning

Level: Undergraduate / College

 Copy Download .txt Print Clean Plain-Text

1 SUMMARY

Reinforcement Learning is a computational approach to machine learning where an agent learns optimal behavior through trial and error interactions with an environment. By receiving numerical rewards based on its actions, the agent seeks to develop a policy that maximizes the total expected return over time. This framework relies on the Markov Decision Process to model sequential decision-making, balancing the need to explore unknown actions against the necessity of exploiting known high-reward strategies.

2 KEY CONCEPTS

1. Markov Decision Process: A mathematical framework consisting of states, actions, transition probabilities, reward functions, and a discount factor used to model decision-making environments.
2. Bellman Optimality Equation: A recursive relationship that defines the value of a state as the maximum expected return achievable from that state by choosing the best possible action.
3. Exploration vs Exploitation: The fundamental challenge of deciding whether to try new actions to gain information or to perform actions known to yield high rewards.
4. Q-Learning: A model-free reinforcement learning algorithm that learns the value of an action in a particular state without requiring a model of the environment transitions.

3 IMPORTANT POINTS

1. The discount factor γ determines the present value of future rewards, where a value closer to zero prioritizes immediate rewards and a value closer to one prioritizes long-term gains.
2. The return G_t is calculated as the sum of discounted future rewards, which prevents infinite sums in continuing tasks.
3. Epsilon-greedy exploration is a common strategy where the agent chooses a random action with probability ϵ and the best-known action with probability $1 - \epsilon$.
4. Q-learning is classified as an off-policy algorithm because it learns the value of the optimal policy independently of the agent's current exploration behavior.

4 EXAM TAKEAWAYS

1. Memorize the MDP tuple (S, A, P, R, γ) as it forms the basis for all reinforcement learning problem definitions.
2. Understand that the Q-learning update rule uses the difference between the target value and the current estimate, scaled by the learning rate α .
3. Be prepared to explain why the discount factor is necessary for convergence in tasks with infinite horizons.
4. Distinguish between model-based approaches that know the transition function P and model-free approaches like Q-learning that do not.

5 IMPORTANT TERMINOLOGY

1. State Space: The set of all possible configurations or situations the agent can encounter in the environment.
2. Action Space: The set of all possible moves or decisions available to the agent at any given state.
3. Policy: A strategy or mapping from states to actions that the agent uses to determine its behavior.
4. Learning Rate: A parameter α that determines the extent to which new information overrides old information during value updates.

6 POSSIBLE EXAM QUESTIONS

1. How does the discount factor γ influence the agent's preference for immediate versus future rewards?
2. Explain the Q-learning update rule and identify the role of the temporal difference error in the formula.
3. Why is the exploration versus exploitation trade-off critical for an agent to perform well in an unknown environment?
4. What are the components of a Markov Decision Process and how do they interact to define the agent's learning environment?

TC05	Empty input
------	-------------

Output:

 **Study Notes Input**

Provide your subject and study material below

1. SUBJECT / COURSE *

DBMS

2. ACADEMIC LEVEL

Undergraduate / College

3. INPUT METHOD *

 Paste Text

 Upload File

STUDY NOTES CONTENT

0 characters

Paste your lengthy study notes here...

 Please enter or upload study material.

 Generate Summary

Clear

Error Handling

- Empty input is detected before an API request.
- Invalid or unsupported input produces a user-friendly message.
- API failures are caught without exposing Python tracebacks.
- Empty or unexpected model responses are handled.
- Very large inputs can be restricted or processed using chunking.

Results and Discussion

The application successfully converts academic study material into structured examination-oriented revision notes. The output helps students identify important concepts, key facts, terminology, exam takeaways, and possible questions. The test cases demonstrate successful processing of different subjects, short input, and invalid empty input.

Limitations

- Summary quality depends on input quality.
- Very large documents may require chunking.
- The model may occasionally omit a minor but important detail.
- Generated summaries should be verified before examination use.
- API quotas and network connectivity can affect performance.

Future Enhancements

- PDF/DOCX support.
- OCR for scanned notes and images.
- Automatic flashcard and quiz generation.
- Downloadable PDF revision notes.
- Multilingual summarization.
- Chapter-wise summarization for large documents.

Conclusion

The AI Student Notes Summarizer was successfully developed using Google Gemini. It provides a simple interface for entering or uploading study material and generates concise examination-oriented revision content. The project demonstrates prompt engineering, API integration, validation, error handling, structured output generation, and testing.

MACHINE TRANSLATION APPLICATIONS

28. English-to-Tamil Translation Application – Develop a machine translation application that translates English text into Tamil. Provide a user-friendly interface and demonstrate the system using different types of sentences.

Problem Statement

English is widely used in education and communication, while many users are more comfortable with regional languages such as Tamil. The objective is to develop an AI-powered English-to-Tamil translation system that accepts English text and generates a natural Tamil translation using Google Gemini.

Objectives

- Develop a machine translation application using Python and Google Gemini.
- Translate English text into natural and meaningful Tamil.
- Provide a simple user-friendly interface.
- Preserve original meaning and context.
- Handle simple, complex, conversational, and technical sentences.
- Handle empty, invalid, unexpected, and API-related errors.
- Test the translator using multiple input cases.

AI Model and Technique

Google Gemini is used as the Large Language Model for translation. It was selected because it can understand context, sentence structure, and technical language. A structured translation prompt instructs the model to preserve meaning, produce natural Tamil, and avoid unnecessary explanations.

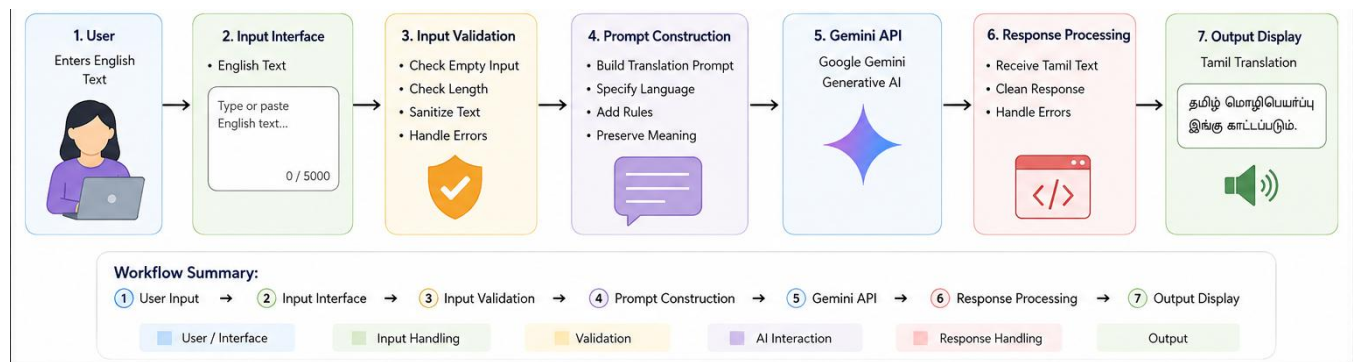
Technologies and Tools Used

1. Python
2. Google Gemini API
3. Google GenAI SDK
4. Web-based user interface
5. Web browser

Application Features

- English text input.
- Tamil translation output.
- Character and word count.
- Translate button.
- Clear button.
- Example sentences.
- Input validation.
- Gemini API integration.
- User-friendly error handling.

Application Architecture and Workflow



User Interface

The application uses a single main page. The left side provides English input and the right side displays the Tamil translation. Translate and Clear buttons provide the main actions.

English → Tamil AI Translator
AI-powered English to Tamil machine translation using Google Gemini

English → Tamil (தமிழ்)

ENGLISH INPUT Words: 0 | Characters: 0

Enter English text here...

Maximum input limit: 5,000 characters

TAMIL TRANSLATION

Tamil translation will appear here after clicking Translate.

Target: Tamil (தமிழ்)

Translate to Tamil Clear

TRY AN EXAMPLE Click any sentence to fill the input

SIMPLE SENTENCE "Artificial intelligence is changing the world."

QUESTION "What time does the college library open?"

COMPLEX SENTENCE "Although the weather was difficult, the students completed their project successfully."

CONVERSATIONAL SENTENCE **TECHNICAL / EDUCATIONAL SENTENCE**

Caption: User interface of the English-to-Tamil AI Translator.

Pseudocode

```
BEGIN
Initialize Google Gemini API
Display Translator interface
INPUT English text
IF text is empty
    DISPLAY validation message
    STOP
END IF
Validate text length
Construct translation prompt
```

```

Send prompt to Gemini API
IF request succeeds
    Receive Tamil translation
    Display translated text
ELSE
    Display user-friendly error
END IF
END

```

Representative Source Code

The complete source code should be submitted separately. The following shows the Gemini API interaction.

```

from google import genai

client = genai.Client(api_key=API_KEY)

response = client.models.generate_content(
    model="YOUR_SUPPORTED_GEMINI_MODEL",
    contents=prompt
)

translation = response.text

```

The API key must not be hard-coded in a publicly shared source file. Store it securely using an environment variable or secret configuration.

Test Cases

Test Case	Input	Expected Result	Status
TC01	Artificial intelligence is changing the world.	Natural Tamil translation preserving meaning	PASS
TC02	What time does the college library open?	Tamil translation of the question	PASS
TC03	Although the weather was difficult, the students completed their project successfully.	Context-preserving Tamil translation	PASS
TC04	Machine learning algorithms learn patterns from data and use them to make predictions.	Technical Tamil translation preserving meaning	PASS
TC05	Empty input	Validation message requesting English text	PASS

Test Case Screenshots

TC01	Artificial intelligence is changing the world.
------	--

Output:

● ENGLISH INPUT

Words: 6 | Characters: 46

Artificial intelligence is changing the world.

Maximum input limit: 5,000 characters

● TAMIL TRANSLATION

[Listen](#) [Copy](#)

செயற்கை நுண்ணறிவு உலகை மாற்றிக்கொண்டிருக்கிறது.

Target: Tamil (தமிழ்)

🕒 Translation completed successfully.

🔄 Translate to Tamil

🔄 Clear

TC02	What time does the college library open?
------	--

Output:

● ENGLISH INPUT

Words: 7 | Characters: 40

What time does the college library open?

Maximum input limit: 5,000 characters

● TAMIL TRANSLATION

[Listen](#) [Copy](#)

கல்லூரி நூலகம் எத்தனை மணிக்கு திறக்கும்?

Target: Tamil (தமிழ்)

🕒 Translation completed successfully.

🔄 Translate to Tamil

🔄 Clear

TC03	Although the weather was difficult, the students completed their project successfully.
------	--

Output:

● ENGLISH INPUT

Words: 11 | Characters: 86

Although the weather was difficult, the students completed their project successfully.

Maximum input limit: 5,000 characters

● TAMIL TRANSLATION

Listen Copy

வானிலை கடினமாக இருந்தபோதிலும், மாணவர்கள் தங்களது திட்டப்பணியை வெற்றிகரமாக முடித்தனர்.

Target: Tamil (தமிழ்)

Translation completed successfully.

Translate to Tamil

Clear

TC04	Machine learning algorithms learn patterns from data and use them to make predictions.
------	--

Output:

● ENGLISH INPUT

Words: 13 | Characters: 86

Machine learning algorithms learn patterns from data and use them to make predictions.

Maximum input limit: 5,000 characters

● TAMIL TRANSLATION

Listen Copy

இயந்திரக் கற்றல் நெறிமுறைகள் (Machine learning algorithms) தரவுகளிலிருந்து அமைப்புகளைக் கற்றுக்கொண்டு, அவற்றைப் பயன்படுத்தி கணிப்புகளைச் செய்கின்றன.

Target: Tamil (தமிழ்)

Translation completed successfully.

Translate to Tamil

Clear

TC05

Empty input

Output:

 Please enter an English sentence to translate.

● ENGLISH INPUT

Words: 0 | Characters: 0

Enter English text here...

Maximum input limit: 5,000 characters

● TAMIL TRANSLATION

Tamil translation will appear here after clicking Translate.

Target: Tamil (தமிழ்)

🔄 Translate to Tamil

🔄 Clear

Error Handling

- Empty input is detected before an API request.
- Input-length restrictions can prevent excessive requests.
- API failures are caught and displayed using a user-friendly message.
- Unexpected or empty model responses are handled.
- Python traceback information is not exposed to normal users.

Results and Discussion

The English-to-Tamil AI Translator successfully translated simple, question-based, complex, and technical English sentences into Tamil. The empty-input test also demonstrated appropriate validation.

Limitations

- Quality may vary for highly complex or ambiguous sentences.
- Idioms and culturally specific expressions may require additional context.
- Some technical terms have no direct Tamil equivalent.
- Generated translations may require human review.
- API quotas and network connectivity can affect performance.

Future Enhancements

- Tamil-to-English translation.
- Additional Indian languages.
- Automatic language detection.
- Document translation.

- Voice input and text-to-speech.
- Downloadable translated documents.
- Multilingual conversational translation.

Conclusion

The English-to-Tamil AI Translator was successfully developed using Google Gemini. The application provides a simple interface for entering English text and receiving a Tamil translation. It demonstrates Generative AI API integration, prompt engineering, natural-language processing, validation, error handling, and testing.

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Model/Technique Fit	30%	Correct, appropriate model/technique choice for the chosen application	Mostly appropriate choices made	Some inappropriate technique choices	Technique choice fundamentally flawed
Functional Completeness	25%	Application fully functional; solves a real use-case effectively	Mostly functional application	Partially functional application	Application is non-functional
Robustness & Reliability	20%	Handles edge cases; stable, reliable performance	Handles common cases well	Limited robustness to edge cases	Frequent failures during use
UI / Demo Polish	15%	Polished UI/demo with an engaging walkthrough	Clear demo, minor polish issues	Basic or rough demo	No usable demo presented
Design & Usage Documentation	10%	Comprehensive documentation of design & usage	Adequate documentation	Minimal documentation	No documentation provided

Marks Evaluation:

Question No.	Model/Technique Fit (30%)	Functional Completeness (25%)	Robustness & Reliability (20%)	UI / Demo Polish (15%)	Design & Usage Documentation(10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						